Documentación Final — Compilador C-TDS

Taller de Diseño de Software (Cod. 3306) — 2026

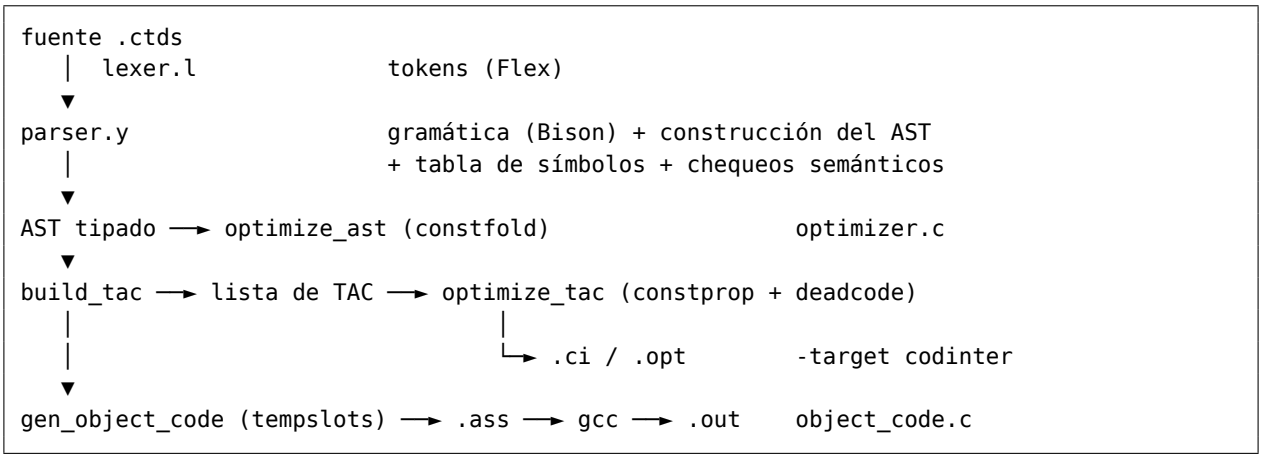
Aguirre Sofía Trinidad, Astrada Camila, Buzzacchi Victoria

Resumen del proyecto. Cada punto remite a la documentación de su etapa que se encuentra en el repositorio. En cada documentación se encuentra el detalle, los ejemplos y los tests que se usaron para verificar la implementación:

Etapa	**Archivo**
1. Análisis léxico y sintáctico	`Documentacion-Etapa1.md`
2. Tabla de símbolos, AST y análisis semántico	`Documentacion-Etapa2.md`
3. Código intermedio (TAC)	`Documentacion-Etapa3.md`
4. Código objeto x86-64	`Documentacion-Etapa4.md`
5. Optimizaciones y correcciones finales	`Documentacion-Etapa5.md`

Uso, build y tests: `README.md`.

Arquitectura



El compilador tiene dos mitades. El **front-end** analiza el programa en una sola pasada. `yyparse()` pide los tokens de a uno a `yylex()`. En cada reducción llama a un constructor del AST (`new_*`), que resuelve los identificadores en la tabla de símbolos y chequea los tipos en ese momento. Al terminar, el resultado es un AST tipado en el que cada identificador apunta a su símbolo.

Si hubo algún error léxico, sintáctico o semántico, el compilador reporta todos los errores encontrados y termina sin generar código. Si no, sigue el **back-end**, que transforma el programa en tres representaciones sucesivas:

- el AST, sobre el que se pliegan constantes;
- el código de tres direcciones (TAC), una lista de instrucciones sobre la que se propagan constantes y se elimina código inalcanzable;
- el assembly x86-64, que `gcc` ensambla y enlaza para obtener el ejecutable.

Las optimizaciones son pasadas opcionales: solo corren las que se activan con `-opt`. `-target` corta la compilación en una etapa y escribe su salida:

- `scan` → `.lex`
- `parse` → `.sint`
- `codinter` → `.ci` (o `.opt`, con optimizaciones)
- `assembly` → `.ass`

Sin `-target` se hace todo el camino hasta el `.out`.

Archivo	Responsabilidad
`lexer.l`	Tokens, comentarios, rango de enteros, símbolos inválidos
`parser.y`	Gramática, precedencia, recuperación de errores, declaración de símbolos y apertura/cierre de niveles
`symbol_table.c/.h`	Pila de niveles; cada nivel es una lista de símbolos
`ast.c/.h`	Nodos del AST y chequeos semánticos al construir cada nodo
`intermediate_code.c`, `tac.h`	AST → lista de instrucciones TAC; impresión del `.ci`
`optimizer.c/.h`	Opción `-opt` y las pasadas de optimización
`object_code.c/.h`	TAC → assembly x86-64 (AT&T, System V)
`main.c`	CLI (`-target`, `-o`, `-opt`, `-debug`) y orquestación de las fases

1. División del trabajo

Etapa	Integrantes
1	Camila: gramática en `parser.y`, números de línea en errores, tests automáticos. Victoria: precedencia y asociatividad. Trinidad: lexer.
2	Camila: AST y conexión del parser con la tabla de símbolos. Trinidad: tabla de símbolos, chequeo de `kind`, cantidad de argumentos, `main`, corrección de tests (Doc. Etapa 2).
3	Victoria
4	Trinidad
5	Camila: correcciones previas (32 bits, Makefile, tests de assembly), `extern` y optimizaciones. Trinidad: corrección guion bajo, retorno en todos los caminos, salidas `scan`/`parse`. Victoria: literal `INT_MIN`, documentación de extensiones, nombres únicos en el TAC.

2. Decisiones, asunciones y extensiones

Solo se nombran las decisiones que difieren de la especificación o que la consigna no deja explícitas. En todo lo demás, el compilador sigue la consigna.

Lenguaje y análisis

- **`while` con paréntesis** (`while (expr) block`), en lugar de `while expr block` como en la gramática de la consigna: consultado en clase. Es la forma de C y la misma que usa el `if` (Doc. Etapa 1).

- **Declaraciones y sentencias en cualquier orden dentro de un bloque**, en lugar de `{ var_decl* statement* }` como en la gramática de la consigna. Con una sola lista de ítems la gramática es más simple, y la regla de "declarado antes de usar" ya la garantiza la tabla de símbolos (Doc. Etapa 1).
- **Recuperación de errores sintácticos** (`error ';'`, `type id_list error`) en lugar de detenerse en el primer error, que es lo que hace bison por defecto. Permite reportar varios errores por corrida. El costo es que puede aparecer un error semántico de más (ver Problemas conocidos) (Doc. Etapa 1 y 2).
- **Retorno en todos los caminos con un chequeo conservador**: un `if` cuenta solo si tiene `else` y las dos ramas retornan, y un `while` nunca cuenta. La alternativa era analizar condiciones constantes (`while (true)`). Se descartó porque el chequeo simple es fácil de verificar y no depende de las optimizaciones. El costo es que puede rechazar programas que en la práctica siempre retornan (Doc. Etapa 5).
- **`int` de 32 bits con wrap**, igual que C, en lugar de reportar el overflow en ejecución. Así coinciden el programa, las funciones C enlazadas con `extern` y el plegado de constantes, que calcula con la misma aritmética (Doc. Etapa 5).
- **`float` sin back-end: error explícito al generar assembly.** Antes se generaba código igual, con los literales en 0, y el ejecutable calculaba mal. Implementar `float` con registros SSE quedó fuera de alcance. Se prefirió un error claro a un resultado incorrecto. El análisis y el TAC sí aceptan `float` (Doc. Etapa 5).
- **No se valida la extensión `.ctds`**, aunque la consigna la pide, para no renombrar los tests, que usan `.txt`. Las salidas se nombran igual con cualquier extensión (Doc. Etapa 5).

Salidas y línea de comandos

- **`.lex` lista el lexema y su línea, no el nombre del token.** La consigna no define el contenido. El lexema alcanza para ver qué reconoció el scanner, y el nombre del token exigiría tocar cada regla de `lexer.l` (Doc. Etapa 5).
- **`.sint` contiene el AST** (`print_ast`: un nodo por línea con sangría, su rol en el padre y su tipo). La consigna permite el árbol o solo el resultado del análisis. Se eligió el árbol porque permite verificar la precedencia y los tipos sin depurar. Con `-debug` o con entrada por stdin también va a la consola.
- **`-opt` escribe un `.opt` aparte** en lugar de reemplazar el `.ci`, para comparar el TAC antes y después de optimizar. (Doc. Etapa 5).
- **La lista de `-opt` es opcional**: el siguiente argumento se toma como lista si no empieza con `-` y no tiene punto. Si no, `-opt` equivale a `-opt all`. Así, un nombre mal escrito (`-opt constflod`) da error en vez de tomarse como archivo de entrada (Doc. Etapa 5).

Extensiones al lenguaje (Doc. Etapa 5)

- **Literales `float` como en C**: `86.`, `.05`, `5e10`. La consigna se contradice: la producción exige parte entera y fraccionaria, pero el texto dice que los literales reales son "iguales a los utilizados en C". Se siguió el texto. Consecuencia: `1.2.3` es un error sintáctico y no léxico.
- **Métodos `extern`**: `int f(int x) extern;` declara una función definida en C, que se enlaza con `c-tds prog.ctds lib.c`. El costo es que `extern` pasa a ser palabra reservada.
- **Literal `-2147483648`**: `2147483648` solo se acepta como operando del `-` unario. No se puede resolver en el lexer, que no sabe si un `-` es unario o una resta. Por eso llega como un token especial y se decide sobre el AST (`resolve_int_min`).

3. Diseño y decisiones clave

Análisis semántico durante el parseo, en lugar de una pasada aparte sobre el árbol terminado. Cada nodo se crea con su tipo ya calculado y cada identificador resuelve su símbolo mientras su nivel está abierto. En una pasada posterior los niveles ya estarían cerrados, y habría que reconstruir los ámbitos para saber a qué `x` se refiere cada uso. Funciona porque la consigna exige declarar antes de usar: al ver un uso, su declaración ya está en la tabla. El costo es que la recuperación de errores sintácticos puede dejar un chequeo incompleto (ver Problemas conocidos). Los errores de tipo se propagan como `TYPE_ERROR` para no reportar en cascada (Doc. Etapa 2).

Tabla de símbolos como pila de niveles, cada uno una lista enlazada, en lugar de una tabla hash. `open_level`/`close_level` siguen la anidación de los bloques, el ocultamiento sale de buscar desde el tope hacia afuera, y para programas de este tamaño la búsqueda lineal alcanza (Doc. Etapa 2).

AST con listas por `next` en lugar de los nodos `NODE_SEQ` del preproyecto, que agregaban un nodo por sentencia sin representar nada del programa. **Un único `NODE_BINOP`** con un `op_type` propio, en lugar de un tipo de nodo por operador: el chequeo de tipos, el TAC y las optimizaciones tratan a todos los operadores igual. Los operadores son propios y no los tokens de bison, así el AST no depende del parser (Doc. Etapa 2).

TAC como lista de instrucciones, en lugar del texto que se escribía en la etapa 3. El generador de assembly necesita saber si cada operando es global, local o parámetro, y extraerlo del texto era frágil. Ahora los operandos guardan su `struct symbol *`, y las optimizaciones trabajan sobre esa misma lista (Doc. Etapa 4).

Nombres únicos para variables homónimas en el TAC (`x`, `x_1`), agregando un sufijo solo cuando dos símbolos distintos de un método se llaman igual. La alternativa era agregar siempre el nivel o un identificador del símbolo, pero el TAC quedaba menos legible. Internamente cada operando se identifica por su símbolo, no por el nombre (Doc. Etapa 3).

Cortocircuito en el TAC con `ifFalse`, `goto` y etiquetas, en lugar de resolverlo en el generador de assembly. Así el generador no trata `&&`/`||` aparte, y las optimizaciones ven los saltos. La etapa 3 evaluaba los dos operandos y se corrigió en la etapa 4 (Doc. Etapa 4).

Código objeto sin asignación de registros: cada variable y cada temporal tiene un slot de 8 bytes en la pila, y cada instrucción TAC se traduce con una plantilla fija. La alternativa, asignar registros (por ejemplo, por coloreo de grafos), produce código más rápido pero es mucho más compleja. Se eligió corrección antes que eficiencia, como recomienda la consigna (Doc. Etapa 4).

`int` de 32 bits sobre slots de 64: se mantienen las instrucciones de 64 bits y cada resultado aritmético se normaliza con `movslq`. También se normaliza el valor que devuelve un `call`: `movslq` para `int` y `movzbq` para `bool`. La alternativa era usar instrucciones de 32 bits (`addl`, `movl`), pero hacía falta una plantilla de carga y guardado distinta según el tipo. Así se cambia una sola instrucción por operación (Doc. Etapa 5).

El `float` se detecta en el AST, no en el generador. En el TAC los temporales no tienen tipo, así que un `float` sin literales (un parámetro `float` llamado con un `int`) no se vería. Además, el chequeo ocurre antes de abrir el `.ass` y no queda un archivo a medio escribir (Doc. Etapa 5).

Optimizaciones como pasadas independientes, no dentro de `new_binop` ni de `gen_expr`. Así se activan por separado con `-opt`, el análisis semántico y sus mensajes no dependen de ellas, y el orden entre pasadas queda explícito en un solo lugar (Doc. Etapa 5).

Propagación de constantes local, por bloque básico: la tabla se vacía en cada etiqueta. La alternativa, un análisis de flujo de datos sobre el grafo de bloques, es más precisa pero más compleja. Repetir `constprop` y `deadcode` hasta un punto fijo recupera parte de lo que se pierde: al borrar código muerto desaparecen etiquetas (Doc. Etapa 5).

Optimización	Sobre	Referencia
`constfold`: plegado de constantes	AST	Doc. Etapa 5
`constprop`: propagación de constantes	TAC	Doc. Etapa 5
`deadcode`: código inalcanzable	TAC	Doc. Etapa 5
`tempslots`: reutilización de slots de temporales	assembly	Doc. Etapa 5

4. Detalles de implementación

- **Línea de los errores con `%locations`**: bison lee un token de más antes de reducir, así que la línea del último token leído podía ser la siguiente. Cada token guarda su línea en `yylloc` y las reglas usan `@1.first_line` (Doc. Etapa 2).
- **`INT_MIN`**: `2147483648` llega como el token `CONSTINT_MIN` y `resolve_int_min` lo convierte en literal solo si es el operando directo de un `-` unario (Doc. Etapa 5).
- **Plegado sin reemplazar nodos**: el nodo plegado se transforma en literal en el mismo lugar, así el padre sigue apuntando a él y se conservan su `line` y su `next` (Doc. Etapa 5).
- **`constprop` y `deadcode` hasta un punto fijo**: cada una habilita a la otra, así que se repiten hasta que una vuelta no cambia nada. Siempre termina porque ninguna deshace lo que hace la otra (Doc. Etapa 5).
- **`tempslots`**: calcula la vida de cada temporal y, en cada instrucción, libera los slots que dejan de usarse antes de asignar los nuevos. El frame queda del tamaño del máximo de temporales vivos a la vez (Doc. Etapa 5).
- **Borrado en listas con `struct tac **pp`**, en lugar de un puntero a la instrucción anterior. El TAC es una lista enlazada, y `deadcode` borra muchas instrucciones en una pasada: código inalcanzable, `goto` a la instrucción siguiente y etiquetas sin uso. Para borrar una instrucción hay que cambiar el puntero que la apunta, que es la cabeza de la lista o el `next` de la anterior. Con un puntero `prev` esos son dos casos distintos. `pp` guarda la dirección de ese puntero (`&code` al principio, `&anterior->next` después), así que borrar es siempre `*pp = i->next`, sin un caso especial para la primera instrucción (Doc. Etapa 5).

5. Problemas conocidos

Los 84 tests de `make test` pasan. No hay casos de test que fallen.

Problema	Cómo se detectó	Alternativa propuesta
`float` no se compila a assembly (error explícito).	Revisión del generador (los literales se cargaban como 0) y del `.ci`. (Doc. Etapa 3)	Marcar conversiones y traducirlas a assembly con registros SSE
`constprop` es local: pierde información en las etiquetas	Diseño (Doc. Etapa 5)	Análisis de flujo de datos (reaching definitions)
Después de un error sintáctico puede aparecer un error semántico de más ("debe retornar un valor en todos los caminos"), porque la recuperación descartó el `return`	Prueba manual con un `while` mal escrito	No correr ese chequeo en métodos donde hubo un error sintáctico. El programa se rechazaría igual

El resumen dice “Fallo en el análisis sintáctico” también cuando los errores son solo semánticos	Salida de los tests de error	Cambiar el encabezado según el tipo de error y modificar tests
No se valida que la entrada sea `.ctds` ni que no empiece con `-`	Comparación con la consigna	Validar en `main.c` y renombrar los tests

Testing

`make test` (`run_tests.sh`) corre 84 casos:

- **Análisis** (`tests/cases/`, 38 programas, 23 con errores): compara stdout y stderr con su `<nombre>.expected` en `tests/expected/`.
- **TAC** (11 `.ci.expected`) y **TAC optimizado** (4 `.opt.expected`, cada uno solo con las optimizaciones que indica su línea `// opt:`).
- **Ejecución** (`tests/asm/`, 15 programas): ensambla, ejecuta y verifica `// exit:`, `// expect:` (salida de `printInt`/`printBool`), `// error:` y `// frame:`. Cada programa corre **sin optimizar y con `-opt all`**, y los dos resultados tienen que coincidir. Los valores esperados se verificaron contra gcc.
- Hay tiempo límite para compilar y ejecutar, así que un ciclo infinito da `FAIL`.